



Nome: _____

OBSERVAÇÕES:

- Os programas devem ser desenvolvidos utilizando a linguagem de programação C
- Não é permitido o uso de funções que não tenham sido abordadas durante as aulas
- Não serão aceitas soluções que usem funções para manipulação de *strings* (string.h)

QUESTÃO 1) (3,0 PONTOS) Faça uma função que receba uma matriz quadrada $N \times N$ (com N definido pela diretiva `#define`). Para cada coluna da matriz, identifique o maior elemento e troque-o com o elemento da diagonal principal que está na mesma coluna. A operação deve ser realizada diretamente na matriz original. A função deve funcionar corretamente para qualquer valor de N .

Exemplo Matriz original (4x4):

$$M = \begin{bmatrix} 1 & 5 & 2 & 7 \\ 7 & 2 & 8 & 1 \\ 4 & 6 & 9 & 0 \\ 3 & 1 & 7 & 5 \end{bmatrix}$$

Matriz resultante:

$$M = \begin{bmatrix} 7 & 5 & 2 & 5 \\ 1 & 6 & 8 & 1 \\ 4 & 2 & 9 & 0 \\ 3 & 1 & 7 & 7 \end{bmatrix}$$

A função obrigatoriamente deverá seguir o protótipo:

```
void troca_maior_coluna_diagonal(int mat[N][N])
```

QUESTÃO 2) (3,5 PONTOS) Um código de rastreamento de encomenda é composto por 10 caracteres. Os 9 primeiros caracteres podem ser letras maiúsculas (A–Z) ou números (0–9), e o 10º caractere é o dígito verificador.

O cálculo do dígito verificador começa com a conversão de caracteres. Cada letra maiúscula do código é transformada em um número correspondente à sua posição no alfabeto, de forma que A equivale a 1, B a 2, C a 3, e assim por diante até Z, que equivale a 26. Os números de 0 a 9 mantêm seu valor original. Em seguida, cada um dos nove primeiros caracteres convertidos é multiplicado por um peso que decresce de 10 até 2, seguindo a ordem dos caracteres.

Por exemplo, no código "A1B2C3D4E", as letras A, B, C, D e E são convertidas nos números 1, 2, 3, 4 e 5, enquanto os números 1, 2, 3 e 4 permanecem com seus valores originais. Multiplicando cada caractere pelos pesos decrescentes, obtemos os produtos:

$$1 \times 10 + 1 \times 9 + 2 \times 8 + 2 \times 7 + 3 \times 6 + 3 \times 5 + 4 \times 4 + 4 \times 3 + 5 \times 2$$

A soma desses produtos é 120. Em seguida, calcula-se o resto da divisão dessa soma por 11 e subtrai-se esse valor de 11 para obter o dígito verificador. Se o resultado for 10, o dígito verificador é representado pela letra X; caso contrário, é o próprio número obtido. No exemplo, o resto da divisão por 11 é 10, então $11 - 10 = 1$, que é o dígito verificador, completando o código "A1B2C3D4E1".

Com base nessa descrição, **desenvolva uma função** que receba como parâmetro uma *string* contendo os 9 primeiros caracteres do código de rastreamento e retorne um caractere correspondente ao dígito verificador. O dígito verificador deve ser retornado como número de '0' a '9' ou como 'X' caso o cálculo resulte em 10. A função obrigatoriamente deverá seguir o protótipo:

```
char calcular_digito_rastreamento(char codigo[10]);
```

Implemente também o **programa principal** que leia a string do código, chame a função `calcular_digito_rastreamento` e exiba o valor retornado pela função.

QUESTÃO 3) (3,5 PONTOS) Considere que em uma empresa os produtos vendidos são armazenados em um vetor de estruturas do tipo TIPOPRODUTO com tamanho N (definido pela diretiva #define). Cada produto possui um código (int codigo), uma descrição (char descricao[51]) e um vetor de vendas mensais (TIPOVENDA vendas[12]). Cada elemento do vetor vendas armazena o mês (int mes), a quantidade vendida (float quantidade) e o preço unitário (float preco_unitario) do produto naquele mês.

```
struct stproduto {
    int codigo;
    char descricao[51];
    TIPOVENDA vendas[MESES];
};
typedef struct stproduto TIPOPRODUTO;
```

Cada elemento do vetor vendas é uma estrutura TIPOVENDA, que armazena o mês (int mes) correspondente à venda, a quantidade de unidades vendidas (float quantidade) e o preço unitário (float preco_unitario) do produto naquele mês. É importante observar que a identificação dos meses nos elementos do vetor de vendas não está necessariamente em ordem crescente; ou seja, o índice 0 do vetor não precisa corresponder a janeiro, o índice 1 a fevereiro e assim por diante. A definição da estrutura TIPOVENDA é a seguinte:

```
struct stvenda {
    int mes;
    float quantidade;
    float preco_unitario;
};
typedef struct stvenda TIPOVENDA;
```

Implemente uma função que, dado o vetor de produtos e um mês específico, retorne a estrutura TIPOPRODUTO correspondente ao produto que obteve a maior quantidade vendida nesse mês. Caso dois ou mais produtos tenham a mesma quantidade vendida no mês, o desempate deve ser feito retornando o produto com o menor código. A função deverá obrigatoriamente possuir o seguinte protótipo:

```
TIPOPRODUTO produto_destaque_mes(TIPOPRODUTO v[N], int mes)
```